Over-synchronization in GPU Programs

Ajay Nayak
Indian Institute of Science
Bengaluru, India
ajaynayak@iisc.ac.in

Arkaprava Basu
Indian Institute of Science
Bengaluru, India
arkapravab@iisc.ac.in

***Abstract*—The performance of GPU (Graphics Processing Unit)-accelerated functions affects a large spectrum of modern software. Efficiently synchronizing across thousands of concurrent threads is critical to the performance of GPU programs. GPU vendors have introduced advanced programming constructs, *e.g., scopes*, for efficiently synchronizing within a chosen subset of threads. However, programmers must explicitly employ them, where applicable, to benefit from such features.**

We demonstrate how GPU programs can leave performance on the table by failing to fully harness advanced synchronization features in modern GPUs – leading to over-synchronization. We discover three different variants of over-synchronization observed in real-world applications. We then build a tool, ScopeAdvice, to find cases of over-synchronization in CUDA programs. Avoiding reported over-synchronization improves the performance of several GPU applications by up to 55%.

***Index Terms*—GPU, synchronization, performance bugs.**

I. Introduction

A broad class of today's software, from deep learning, graph processing, and data analytics to scientific computing, rely on GPU's massively parallel computing [1]–[3]. Consequently, the efficiency of GPU programs has an overwhelming bearing on the performance of a large swath of software.

GPUs significantly contribute to a server's capital and operational costs. We estimate that 87% of the cost of a DGX server with four NVIDIA A100 GPUs is due to the GPUs only [4], [5]. They contribute significantly to the power budget, too [6]. Unfortunately, GPUs are often left under-utilized by the software, significantly but unnecessarily increasing the cost of computing [7]–[11].

To the best of our knowledge, we are the first to demonstrate how sub-optimal synchronization in GPU programs can cause significant performance inefficiency. Many key use cases of GPU, *e.g.,* graph processing, need fine-grain synchronization, *e.g.,* fences. The efficiency of synchronizing across thousands of concurrent GPU threads can significantly affect a GPU program's overall performance. However, GPU's hierarchical programming paradigm and its nuanced synchronization model [12], [13] make writing correct GPU programs with minimal synchronization challenging.

To keep thousands of concurrent GPU threads tractable, they are organized as groups of up to 1024 threads called a threadblock. Synchronizing across all the threads of a kernel is slow and often unnecessary due to GPU's hierarchical programming paradigm. Programming languages like CUDA and OpenCL support *scoped* synchronization to write efficiently synchronized code. The *scope* qualifiers guarantee that the effect of the synchronization is visible *only* within a subset of threads (*i.e.,* its scope). Those with a *narrower* scope that synchronizes a smaller subset of threads are faster. For example, on an NVIDIA RTX 3090 GPU, the block-scope fence that guarantees visibility in the issuing thread's threadblock is $21\times$ faster than the device-scope fence that ensures visibility across all threads of a kernel.

The default scope in both CUDA and OpenCL is the device scope. Programmers must explicitly use a *narrower* scope (*e.g.,* block), where applicable, for good performance. However, a data race can manifest if the scope is narrower than needed for synchronizing threads [14]–[16]. Unsurprisingly, programmers often use a wider scope than necessary, prioritizing correctness over performance, even in popular CUDA libraries written by experts (*e.g.,* cuML [17]). A programmer could have used a narrower-scoped operation to improve performance without altering the program's behavior. We call such occurrences *over-synchronization*.

We discover *three* subtle variants of over-synchronization and redundant synchronization in CUDA programs [18] that may leave significant performance on the table. Their subtlety arises from the interaction of CUDA's scoped synchronization model and the GPU micro-architecture.

First, we find that a fence can have a wider scope than necessary due to the nuanced semantics of related memory operations. Assume that a variable is written to (store) and read from (load) by threads from different threadblocks. It is natural to assume that synchronization (*e.g.,* fence) of device scope is needed after the write and before the read to ensure that the latter observes data produced by the first thread. However, we observe that a narrower-scoped fence will suffice *if* the instruction performing the read skips the GPU's L1 cache to directly look up the GPU's L2 cache.[1] For example, a load to a variable declared 'volatile' is guaranteed to skip the L1 cache and return the value from the coherent L2 cache or memory (§14.5.3.3 of [18]). Similarly, the memory operations in a device-scoped atomic read-modify-write instruction also skip the L1 cache. In such scenarios, the fence is needed *only* to ensure ordering amongst the loads/stores and *not* for ensuring visibility of updates across threadblocks. As a fence of *any* scope guarantees ordering, using a narrower-scoped

[1]Unlike CPUs, GPU hardware, by design, eschews cache coherence for its L1 caches to avoid overheads of maintaining coherence across thousands of concurrent loads/stores. The L2 cache, however, is coherent.

fence, *e.g.,* a block-scoped fence, before reading the variable, would have preserved the program semantics (Section III-1).

In another incarnation of over-synchronization, a fence could be entirely redundant. We notice that the threadblock barrier in CUDA (__syncthreads) encompasses the semantics of a block-scoped fence beyond acting as an execution barrier across threads in a threadblock. Thus, using a block-scoped fence immediately following a barrier is redundant. Finally, we notice that common lock/unlock routines may unnecessarily synchronize across all threads of a kernel, even when the communicating threads fall within the same threadblock.

We find evidence of these variants of over-synchronizations across popular GPU-accelerated libraries and open-source programs, including cuML [17], cudpp [19], cuBLAS [20], and gpufilter [21]. In each case, modifying only *three* lines of code on average, *i.e.,* either by changing a fence's scope or removing one, improves the performance by up to 55% without affecting program behavior.

We demonstrate how removing these over-synchronizations retains the original program's semantics. NVIDIA publishes only a programming optimization guide for CUDA but does not formally specify its memory consistency model. However, it formally specifies the memory model for PTX [22]. All programs written in CUDA code are lowered to PTX. We demonstrate that the removal of over-synchronizations is also valid against the formal PTX memory consistency model.

Unfortunately, given the subtleties of these over-synchronizations, it could be challenging to manually notice these, as evidenced by their existence in libraries written by experts. Our final contribution thus is ScopeAdvice, a tool that detects over-synchronization in CUDA programs. It uses NVIDIA's NVBit library [23] to trace a kernel's execution and gather relevant memory access and synchronization traces for analysis. ScopeAdvice reports the line number, source file, and the type of over-synchronization (if present). Given concrete, actionable advice, the programmer can enhance their code to improve performance.

Beyond its usefulness to existing programs, ScopeAdvice eases programming too. Scoped synchronization significantly burdens programmers [24], [25]. With ScopeAdvice, programmers can first focus on functional correctness while writing new programs, possibly with over-synchronization, and then use ScopeAdvice to find optimization opportunities.

Over-synchronizations will likely become more prevalent as GPU vendors continue to introduce newer levels in the programming hierarchy and, thus, newer synchronization scopes. For example, in Hopper GPUs, NVIDIA introduced a new level in the hierarchy called a *cluster*, encompassing up to eight threadblocks sharing a memory region (§2.2.1 of [18]). The associated new synchronization scopes further increase the possibility of introducing over-synchronizations.

In summary, we make the following contributions.

- We are the first to report sub-optimal performance of GPU programs due to over-synchronization. We find *three* variants of over-synchronization.
- We demonstrate that removing over-synchronization from CUDA programs does not alter the program's semantics.
- We built a runtime tool, ScopeAdvice, to detect and report suspected cases of over-synchronization to the programmer. Following the tool's advice, programs sped up by up to 55%.

II. Background

***GPU hierarchy*:** GPUs arrange thousands of concurrent threads in a hierarchy. The top of the hardware hierarchy consists of Streaming Multiprocessors (SM). Each SM sports multiple Single-Instruction-Multiple-Data (SIMD) units further divided into execution lanes. All SIMD units in an SM share a private L1 cache and scratchpad (shared memory). All the SMs share a larger L2 cache. The GPU's global memory (on-board device memory) is accessible to all GPU threads. The hardware caches contents of global memory in L1 and L2 caches. GPU hardware does not keep its L1 cache contents coherent to avoid excessive overheads across hundreds of SMs. However, L2 cache contents are always coherent.

In the software hierarchy, the smallest execution unit is a thread mapped to an execution lane in a SIMD unit. A group of threads (typically 32) forms a warp, the smallest hardware-scheduled unit of work that executes on SIMD units. A block (threadblock) is a group of warps that executes on the same SM. Threads in a threadblock can communicate amongst themselves via scratchpad. Finally, a grid comprises all the threadblocks used for executing a GPU kernel.

***Scoped Operations on GPUs*:** Global synchronization across all the threads on a GPU is often unnecessary. Since the execution is naturally divided into threadblocks, synchronization within a threadblock is often sufficient. NVIDIA, AMD provides a programming construct called *scopes* that enables specifying the subset of the threads that should synchronize.

CUDA supports scope qualifiers for fence and atomic operations. Fences in CUDA provide both ordering and visibility guarantees. A fence of *any* scope ensures that memory instructions in the issuing thread cannot be reordered across the fence. The scope qualifier of the fence ensures the visibility of operations across a subset of threads. For example, a block-scoped fence (__threadfence_block) guarantees that all writes by the issuing thread are made visible to all the threads in its threadblock. In contrast, a device-scoped fence (__threadfence, default) must ensure that the writes before it are visible across all the threadblocks. This requires flushing dirty cache lines from the private, incoherent L1 cache to the coherent L2. The device scope must ensure that the reads after the fence get the data from the L2 cache, which requires invalidating the L1 cache contents. In comparison, a block-scoped fence must ensure visibility only within a threadblock. It does not require flushing or invalidating L1 cache contents since all threads in a threadblock share the same L1 cache. The system scope ensures visibility across all GPUs (multi-GPU system) and CPU threads. We focus on execution on a single GPU and, thus, limit our discussion on block and device scopes.

Atomic read-modify-write (RMW) instructions can also be qualified with scopes. Block-scoped atomics ensure visibility

```
                    (global) volatile *flags; (global) *sum, *status;
1   volatile Flags *mFlag = flags + blockIdx.x;
2   mFlag->sum = SUM;
3   __threadfence();
4   mFlag->status = 1;
5   ...
6   volatile Flags *sFlag = flags + (blockIdx.x - 1);
7   while (sFlag->status == 0);
8   __threadfence(); // _block() is sufficient
9   accumulate += sFlag->sum;
```

(a) In the presence of volatile variables.

```
1   oldVal = atomicAdd(sum, SUM);
2   __threadfence(); // _block() is sufficient
3   atomicExch(status, 1);
4   ...
5   while (!atomicCAS(status, 1, 0));
6   __threadfence(); // _block() is sufficient
7   accumulate += atomicAdd(sum, 0);
```

(b) In the presence of device-scoped atomic RMWs.

Fig. 1: Over-synchronization due to accesses skipping L1$.

across all threads within a threadblock. Since threads in a threadblock share an L1 cache, block-scoped atomics can be completed in the L1 cache. A device-scoped atomic ensures visibility across all threadblocks. Thus, it *must skip the L1 cache* and read from/write to the coherent L2 cache shared by all the threads of a kernel (§8.4.1 of [12]). Further, atomics in CUDA follow *relaxed* semantics, *i.e.,* loads/stores can bypass atomics (§7.14 of [18]). Hence, lock/unlock routines in CUDA are implemented as a combination of an atomic and a fence.

CUDA also provides barrier (__syncthreads) instruction that ensures all threads of issuing thread's threadblock reach the barrier before proceeding further. It encompasses the semantics of a __threadfence_block – writes by any thread of a threadblock before the barrier is visible to all threads in that threadblock after the barrier.

***Impact of volatile keyword*:** The volatile qualifier for variables in CUDA impacts their visibility. Access to volatile variables skips the L1 cache and returns data from the GPU's L2 cache (§14.5.3.3 of [18]). Consequently, a block-scoped fence is sufficient to order reads of two volatile variables even when threads from different threadblocks access them.

III. Over-synchronization in GPU programs

Programmers may use a wider scope in fences than needed or add a redundant fence to be *safe* from introducing data races [15], [16]. We call this over-synchronization, *i.e.,* occurrences where using a narrower scoped fence would *not* have introduced a data race (bug) and preserved the program's behavior. A program's behavior is preserved by ensuring that all store-to-load relations remain unaltered, *i.e.,* the stores whose output a load can return are the same in the original program and its version without over-synchronization.

We analyze open-source CUDA programs and libraries to investigate how over-synchronization occurs in practice. We find *three* distinct variants of over-synchronization. This analysis leads to understanding code constructs that cause it and drives the tool's design to detect over-synchronization.

1) ***Variant 1****:* Programmers assume that they must use device-scoped fences when communicating across threads from different threadblocks. While generally true, a block-scoped fence is enough *if* the instruction reading the shared variable skips the L1 cache to directly access the L2 cache/memory. This observation holds because the visibility of updates by threads across threadblocks, *i.e.,* values read, is guaranteed by such instructions. Thus, the fence is necessary to ensure *only* the ordering among memory operations in the issuing thread. Since all fences, including the narrowest scope, ensure ordering, a wider-scoped one is unnecessary. We find that such over-synchronizations can manifest in *two* ways based on how the shared variable is read. We use producer-consumer constructs to describe these cases.

***In the presence of volatile variables*:** Consider a simplified snippet (Figure 1a) from the 'decoupledLookback' kernel in cuML [17]. A thread updates the variable sum, followed by a device-scoped fence in lines 2-3. In line 4, the thread updates status, indicating that the updated sum is available. The fence in line 3 is needed to ensure that status is updated after making the updated sum visible to all threads. A thread from another threadblock waits for the status to be updated and then reads the updated sum (lines 7-9).

The device-scoped fence in line 8 seems necessary as the threads accessing sum are from different threadblocks. However, mFlag and sFlag are declared volatile. Note that declaring a pointer to a struct type as volatile (here, mFlag and sFlag) means that the pointer points to a volatile structure. This implies that the entire structure is to be treated as volatile. The loads to volatile variables (status and flag members in the Flags struct type) skip the L1 cache to read the latest value from the coherent L2 cache (§7.5 of [18]). This ensures the visibility of updates made by threads across threadblocks (§14.5.3.3 of [18]). Thus, threads always read the latest values of sum. The fence in line 8 is needed only for ordering the reads of status (line 7) and sum (line 9). Thus, the narrowest scoped fence (here, block-scope) is sufficient.

***In the presence of device-scoped atomic RMWs*:** The memory operations (read/write) of a device-scoped atomic RMW behave similarly to those of a volatile variable – skip the L1 cache to access the latest data from the L2 cache. Thus, an accompanying fence is needed only for ordering.

Consider the code snippet in Figure 1b. A thread updates the variable sum using a device-scoped atomicAdd, followed by a device-scoped fence in lines 1-2. In line 3, the thread updates the variable status using a device-scoped atomicExch, indicating that the updated data is visible to all threads. A thread from another threadblock waits for the status to be updated and then reads the sum (lines 5-7) using device-scoped atomicCAS and atomicAdd, respectively.

The device-scoped fences in lines 2 and 6 seem necessary as the threads accessing data are from different threadblocks. However, the snippet uses device-scoped atomic RMWs, ensuring that all threads observe the latest value. The fences in lines 2 and 6 are needed only for ordering the atomics in lines 1, 3, and lines 5, 7, respectively. The atomicity guarantee

```
    (stack) *saveValue, *Aval; (shared) *addressPad
1   ...
2   for (i = 0; i < 8; i++)
3     saveValue[i] = addressPad[tid*depth+i];
4   __threadfence();  // redundant
5   __syncthreads();
6   for (i = 0; i < 8; i++)
7     addressPad[Aval[i]] = saveValue[i];
8   ...
```

Fig. 2: Redundant fence due to barrier.

```
1   // implementing lock acquire
2   while(atomicCAS(lock, 0, 1) != 0);
3   __threadfence(); // _block() for intra-threadblock lock
4
5   // implementing lock release
6   __threadfence(); // _block() for intra-threadblock lock
7   atomicExch(lock, 0);
```

Fig. 3: Over-synchronized *intra*-threadblock locking.

requires that once the read to an atomic variable completes, there cannot be any intervening access before the write part of the atomic. Therefore, ensuring that the reads of atomic RMWs are ordered guarantees the ordering of atomics. The block-scoped fences are sufficient to ensure this.

*2) **Variant 2:*** In CUDA, barriers implicitly contain the semantics of a block-scoped fence (§7.6 of [18]). Besides ensuring that threads in a threadblock reach the barrier, it guarantees that writes before the barrier are visible to reads from threads in that threadblock after the barrier. Thus, a block-scoped fence immediately neighboring a barrier is redundant.

Figure 2 shows a code snippet from the sort application in the cudpp library [19]. A thread reads from addressPad in the shared address space (scratchpad) and writes to the local variable saveValue (registers or thread-local memory). This is followed by a device-scoped fence and a barrier. Next, saveValue is written back to addressPad, depending on the content of Aval. Note the double indirection in the accesses to addressPad in lines 3 and 7. The barrier (line 5) ensures that the read in line 3 can race with the write in line 7.

It would seem that the programmers intended to guarantee ordering using a device-scoped fence on line 4. However, a block-scoped fence would suffice since threads within a threadblock are communicating via the scratchpad. Notably, the barrier (line 5) immediately following the over-synchronized device-scoped fence (line 4) makes the fence *redundant*. Therefore, removing the fence does not alter the program's behavior or introduce race.

*3) **Variant 3:*** Applications often use lock (acquire) and unlock (release) routines for synchronization. In CUDA, these routines are implemented with a sequence of fences and atomics [13], [26], [27]. While synchronizing among threads from different threadblocks necessitates device-scoped operations, block-scope is sufficient if the communicating threads belong to a threadblock. Using the same routine for lock or unlock makes them over-synchronized when communicating threads do *not* straddle threadblocks.

Figure 3 shows a typical code snippet implementing acquire-release for fine-grain locking. For better performance, one must use different scoped operations based on the communicating threads. A block-scoped fence in Figure 3 would have ensured the ordering and visibility of memory operations inside the acquire-release critical section amongst threads within a threadblock (§7.5 of [18]). Thus, fences in lines 3 and 6 are over-synchronized.

IV. Validation against PTX memory model

Section III reports over-synchronization as per the CUDA programming guide. However, CUDA does not formally specify a memory model. Instead, NVIDIA has a formally specified memory model for PTX [12], [22]. PTX is a virtual instruction set for NVIDIA GPUs. CUDA programs are first lowered to the PTX, which is then lowered to SASS or binary that runs on the GPU. It is natural to wonder if our proposed optimizations of removing over-synchronization are *valid* as per the PTX memory model. Removal of an over-synchronization is valid if it: ① does not introduce data races, and ② preserves the original program behavior. The program's behavior is preserved if a load can return only the values generated by the same set of stores as in the original program.

***Relevant background on PTX memory model*:** CUDA can be easily mapped to the PTX instructions. Accesses to volatile variables in CUDA are suffixed with '.volatile' in PTX. Atomics are prefixed with 'atom' and specify scope as a substring, *i.e.,* '.cta,' '.gpu,' and '.sys' for block, device, and system scopes, respectively. A __threadfence is lowered to a membar instruction suffixed with its scope. The barrier (__syncthreads) is lowered to bar.sync. We describe a few definitions from the PTX memory model that are relevant to our discussion:

- **Morally strong memory operations** (§8.6 of [12]). Memory operations from different threads are defined as morally strong when: ① they are performed on the same address, and ② they have a scope that includes the involved threads. For example, a read and a write from threads of different threadblocks are morally strong if they are performed on the same address and the operations are device or system-scoped.
- **Data races** (§8.6.1 of [12]). The PTX model defines a data race as a set of conflicting operations, with at least one write, which are *not morally strong and* are not ordered.
- **Program order** (§8.8.1 of [12]) is the sequential order of all operations performed by a thread.
- **Observation order** (§8.8.2 of [12]) relates morally strong read, write originating from different threads where the value read is the value written by the write.
- **Synchronization order** (§8.8.4 of [12]) specifies the scope qualifier for ordering the memory operations from different threads. For example, a device-scoped fence establishes the synchronization order of memory operations from threads of different threadblocks.

A. Validity of Variant 1 over-synchronization

The example programs in Variant 1 (Figure 1) follow the producer-consumer construct. The programmer expects that the consumer reads the value written by the producer

```
(global) sum = 0, flag = 0;
1   W1 st.volatile [sum], SUM;          // sum = SUM;
2   F1 membar.gl;                       // __threadfence();
3   W2 st.volatile [flag], 1;           // flag = 1;
4   ...
5   LB_1:
6   R1 ld.volatile %r1, [flag];         // while (flag == 0);
7      setp.eq %p1, %r1, 0;             //
8      @%p1 bra LB_1;                   //
9   F2 membar.cta;                      // __threadfence_block();
10  R2 ld.volatile %r2, [sum];          // aggregate += sum;
```

(a) PTX representation of program in Figure 1a.

```
1   A1 atom.gpu.add %r2, [sum], SUM;    // atomicAdd(...);
2   F3 membar.cta;                      // __threadfence_block();
3   A2 atom.gpu.exch %r3, [flag], 1;    // atomicExch(...)
4   ...
5   LB_2:
6   A3 atom.gpu.cas %r1, [flag], 1, 0;  // while (!atomicCAS(...))
7      setp.eq %p1, %r1, 0;             //
8      @%p1 bra LB_2;                   //
9   F4 membar.cta;                      // __threadfence_block();
10  A4 atom.gpu.add %r6, [sum], 0;      // atomicAdd(...);
```

(b) PTX representation of program in Figure 1b.

Fig. 4: Simplified PTX representation of programs in Figure 1 after eliminating Variant 1 over-synchronization.

after the program's execution. Thus, the elimination of over-synchronization are valid if they satisfy *two* requirements: ① there are no data races in the optimized code, and ② the consumer receives the value written by the producer.

1) ***Eliminating over-synchronization in the presence of volatile variable****:* Figure 4a is a simplified PTX representation of the program from Figure 1a *after* removing the Variant 1 over-synchronization. The inline comments provide reference to the corresponding CUDA syntax. The instructions in lines 1-3, Figure 4a, map to lines 2-4, Figure 1a. The instructions in lines 6-10, Figure 4a, map to lines 7-9, Figure 1a.

No data race**:** Note that all memory operations in the program are volatile. Volatile operations are equivalent to 'relaxed.sys' memory operations (system scope) as per the PTX memory model (§9.7.8.8 of [12]). Thus, all pairs of memory operations accessing the same address are morally strong. Morally strong operations cannot cause a data race (§8.6.1 of [12]).

A block-scoped fence in the consumer ensures latest value**:** The load at R2 must read the value written at W1 if R1 reads the value written at W2. Note that the loop (Figure 4a, lines 6-8) ensures that the consumer does not proceed till it reads the value of 1 for flag, *i.e.,* the observation order W2→R1. There can only be two possibilities when the load at R2 may not read the latest value written at W1: ① if the instruction at R2 gets re-ordered (*e.g.,* by the compiler) before that at R1, or ② if the instruction at W1 gets re-ordered after that at W2.

However, the fence instructions at F1 and F2 guarantee sequential program order (§9.7.12.3 of [12]) preventing the above-mentioned possibilities since a fence of any scope guarantees *program order*. The fences ensure program order of W1→F1→W2 in the producer and of R1→F2→R2 in the consumer. Further, F1 ensures that W1 is visible at the device scope before W2 in the same scope. The program order in each thread and the observation order across the threads provide the total ordering of W1→F1→W2→R1→F2→R2. This establishes the observation order W1→R2. Thus, the consumer *always* reads the latest value written at W1 of sum at R2, retaining the original program behavior.

2) ***Eliminating over-synchronization in presence of device-scoped atomic RMWs****:* Figure 4b is a simplified PTX representation of the program from Figure 1b after eliminating Variant 1 over-synchronization. The inline comments provide reference to the corresponding CUDA syntax. The instructions in lines 1-3, Figure 4b, map to lines 1-3, Figure 1b. The instructions in lines 6-10, Figure 4b, map to lines 5-7, Figure 1b.

No data race**:** All memory operations in the program are device-scoped atomic, and thus morally strong. Consider the atomic RMWs A2 and A3 on the flag. These are performed on the same address, and both are device-scoped. The communicating threads fall within the scope as it covers all GPU threads. As all memory operations are morally strong, they cannot cause a data race (§8.6.1 of [12]).

Block-scoped fences ensures latest value**:** The atomic at A4 must observe the effect of A1 if A3 observes the effect of A2. The loop (Figure 4b, lines 6-8) ensures that the consumer does not proceed till it reads the value of 1 for flag, *i.e.,* the observation order A2→A3. There can only be two possibilities when A4 may not observe the effect of A1: ① if A4 gets re-ordered before A3, or ② if A1 gets re-ordered after A2.

However, fences F3 and F4 help avoid these re-orderings since fences of any scope *guarantee* program order (§9.7.12.3 of [12]). The fences ensure program order of A1→F3→A2 in the producer and of A3→F4→A4 in the consumer. The fence F3 ensures that the 'read' in A1 completes before A2 completes. As A1 is an atomic RMW, once the read completes, its 'write' must also complete before any other intervening access to data. The program order in each thread and the observation order across the threads provide the total ordering of A1→F3→A2→A3→F4→A4. Thus, the consumer *always* observes the value written at A1 for data at A4.

3) ***Eliminating redundant synchronization order****:* Thanks to the use of device-scoped fences, the original programs in Figure 1 had synchronization order between the producer and consumer threads. However, in the presence of volatile variables (Figure 4a), R1 and R2 themselves ensure visibility across threadblocks. For device-scoped atomic RMWs (Figure 4b), the visibility is guaranteed by the atomic themselves (A1-A4). Thus, we make a crucial observation that the synchronization order in the original program is *unnecessary* to preserve the program's semantics since fences are not needed for visibility. We leverage this to use block-scoped fences that sacrifice synchronization order in the optimized program.

B. Validity of Variant 2 and Variant 3 over-synchronizations

Eliminating over-synchronizations due to Variants 2 and 3 are trivially valid. In PTX, the elimination of Variant 2 suggests that when a block-scoped fence immediately follows a bar.sync or vice-versa, the block-scoped fence is redundant. The semantics of bar.sync is a superset of a block-scoped fence

(§9.7.12.1 of [12]). Thus, removing the fence cannot alter the program's behavior.

In the PTX representation, eliminating Variant 3 suggests using a block-scoped fence to synchronize threads of a threadblock. A block-scoped fence guarantees visibility of memory operations within a threadblock (§9.7.12.3 of [12]) and, thus, cannot affect the program's behavior.

V. ScopeAdvice: Finding over-synchronization

Manually finding over-synchronization can be challenging, as evidenced by over-synchronizations in libraries written by CUDA experts. Thus we set out to build ScopeAdvice, a tool that automatically highlights cases of over-synchronization in programs. Such tools can also ease programming by allowing programmers to focus on functional correctness. The tool can help find opportunities to optimize performance later.

To infer if a fence is over-synchronized, we need to ascertain its necessity for a program's functional correctness. We *assume* that all fences in a program that are *not* qualified with the narrowest scope (block here) are over-synchronized and establish the necessity of their wider scopes based on memory accesses. First, we map the stream of memory and synchronization accesses onto a *trace* for reasoning [28]. Next, we lay the rules on the trace to detect over-synchronization.

A. Modeling Program Execution as a Trace

We define a *trace* as a stream of operations executed on a GPU, *i.e.,* a dynamic execution trace of a kernel. It captures necessary information to infer if a fence is over-synchronized. The trace contains memory instructions to global memory and relevant synchronization operations. Access to other types of GPU memory, *e.g.,* scratchpad, does not necessitate a fence with a wide scope and, thus, is ignored. The trace contains the following elements:

- *rd(t, x)* or *wr(t, x)*; thread t performs a read or write operation on a global memory address x.
- *rdSkipL1(t, x)*; thread t performs a read on a global memory address x that skips the L1 cache.
- *wrV(t, x)*; thread t performs a write on a global memory address x that is declared volatile.
- *atm(t, x, scope)*; thread t performs a *scoped* atomic RMW operation on a global memory address x.
- *fence(t)*; thread t performs a device-scoped fence.

The trace captures the addresses of load/stores and the thread ID performing it (*e.g., rd(t, x)*). Loads to global memory that skip the L1 cache (*e.g.,* variables declared 'volatile') are captured as *rdSkipL1(t, x)*. Stores to variables declared 'volatile' are captured as *wrV*. For atomic RMWs, the trace captures its scope, the address, and the thread ID. A block-scoped atomic is treated as if *rd* and *wr* are put together, while a device-scoped atomic has an effect of *rdSkipL1* and *wrV* put together. However, unlike *wrV*, a device-scoped *atm* does not need a fence to guarantee visibility across threadblocks. The trace contains device-scoped fences executed by threads. As block-scoped fences have the narrowest scope, they cannot be over-synchronized. Thus, the trace skips them. The trace does not capture barriers either. Static analysis of the source code is enough to infer whether a block-scoped fence is redundant in the presence of a neighboring barrier.

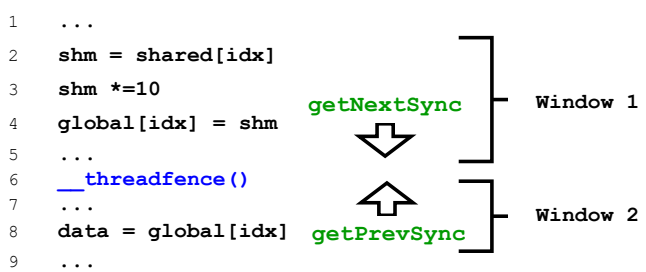

Fig. 5: Example of windows.

B. Relating Memory and Fence Operations

The necessity of synchronization and its scope depends upon the memory accesses and the threads accessing a shared location. Thus, we relate memory accesses to a fence to infer the necessity through *windowing*, similar to prior work [29].

We split the stream of instructions into windows with *fence*s acting as sentinels. A window is a set of instructions between *two fence*s for each thread. We treat the beginning and the end of a kernel to contain a *fence* implicitly. Thus, all instructions are part of a window, even if the kernel has no fences. Windows relate every memory operation that needs synchronization (*rd*, *wr*, *rdSkipL1*, and *wrV*) to their closest fence. For *rd* and *rdSkipL1*, the corresponding *fence* is the one that started the window in which the operation lies. This *fence* affects the value that is read. Similarly, for *wr* and *wrV*, the corresponding *fence* is the one that terminated the window in which the operation lies. This *fence* determines the group of threads to which the write is guaranteed to be visible. We define two routines for the purpose: ① *getPrevSync(t, read)*; returns the preceding fence for a *read* (*rd* or *rdSkipL1*) executed by thread *t*, ② *getNextSync(t, write)*; returns the next fence for a *write* (*wr* or *wrV*) executed by thread *t*.

We illustrate the creation of *windows* with the example in Figure 5. The code is split into two windows; one consists of all instructions until line 6 and one after. The *getNextSync()* for the write in line 4 and the *getPrevSync()* for the read in line 8 returns the fence in line 6.

C. Identifying Over-synchronized Operations

We assume that *every* fence with a scope wider than block, *i.e.,* device and system, to be over-synchronized *unless* its necessity is established by **Rules 1** and **2** below.

***Rule 1*:** Given two operations, *rdSkipL1(t_1, x)* and *wrV(t_2, x)*, such that t_1 belongs to threadblock b_1, t_2 belongs to threadblock b_2, and b_1 != b_2, then: ① the *fence* identified by *getPrevSync(t_1, rdSkipL1)* should be block-scoped, ② the *fence* identified by *getNextSync(t_2, wrV)* should be device-scoped *unless* the *wrV* is due to a device-scoped *atm*. A block-scoped *fence* ensures the ordering of all *rd* and *rdSkipL1* operations. This assurance, and the ones provided by *rdSkipL1*,

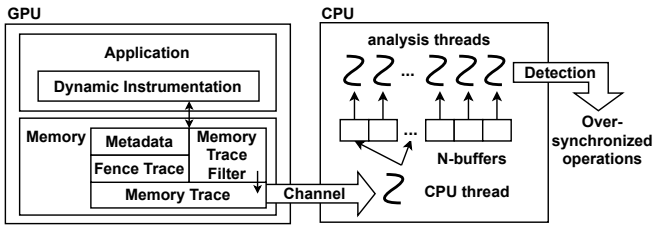

Fig. 6: High-level components of ScopeAdvice.

[64] Address	[1] Load	[1] Store	[5] WindowId	[2] Scope	[23] ThreadId

Fig. 7: A unit of memory execution trace (with bit width).

[1] Valid	[1] Store	[1] MultiBlock	[20] BlockID	[2] Count

Fig. 8: A unit of memory metadata (with bit width).

ensures that the thread reads the latest value written by a thread from a different threadblock [18]. *A device- or system-scoped fence would be over-synchronized.* For *wrV*, a device-scoped fence is necessary to ensure visibility across threadblocks.

***Rule 2*:** Given two operations *rd(t_1, x)* and *wr(t_2, x)*, such that t_1 belongs to threadblock b_1, t_2 belongs to threadblock b_2, and $b_1 == b_2$, then: ① the *fence* identified by *getPrevSync(t_1, rd)* should be block-scoped, ② the *fence* identified by *getNextSync(t_2, wr)* should be block-scoped. This rule effectively says that a block-scoped fence is sufficient if the threads writing to and reading from a shared location belong to the *same* threadblock. *A wider scope (e.g., device or system) would be over-synchronized.*

***Rule 3*:** If a block, device or a system-scoped fence that was found over-synchronized following **Rule 1** or **2** is an immediate neighbor to a barrier in the source code, then the fence is redundant. This rule is not applied to the trace. Instead, it is applied during static source-code analysis of the source after applying **Rules 1** and **2** on the trace.

We demonstrate how these rules detect over-synchronizations in examples from Section III. **Rule 1** captures the examples of **Variant 1**. The fence in line 8 of Figure 1a would be declared over-synchronized due to volatile reads. Similarly, the fences in lines 2 and 6 of Figure 1b would be declared over-synchronized due to the presence of device-scoped atomic operations. To understand how the rules work for **Variant 2**, note that ① the execution trace contains accesses to the global memory *only*, ② all fences are assumed over-synchronized unless proven otherwise by **Rules 1** or **2**. In Figure 2, all accesses are to *non*-global memory (scratchpad and thread-local store). Therefore, those accesses will *not* appear in the trace. Consequently, the fence in line 4 will be declared over-synchronized since **Rules 1** and **2** fail to establish the need for its device scope. This fence will be treated as a block-scoped fence while applying **Rule 3** during the static analysis. That would then be declared redundant. The over-synchronization in Figure 3 (**Variant 3**) is caught by applying **Rule 2** since the accesses to the shared address are from threads of the same threadblock.

Notice that while the trace contains atomic RMWs, the rules are silent on them. While atomic RMWs are needed to track access to global memory, they are treated as a combination of *rd* and *wr* or that of *rdSkipL1* and *wrV*, as discussed earlier.

VI. Implementation of ScopeAdvice

Figure 6 provides an overview of ScopeAdvice's implementation. A CUDA kernel is instrumented to collect the execution trace as it executes on the GPU. ScopeAdvice ships the collected trace and additional metadata to the CPU for analysis. The multi-threaded analyzer on the CPU enforces the rules described in Section V on the execution trace, metadata, and source to identify over-synchronization.

A. Instrumentation and Execution Metadata

We use NVIDIA's NVBit library [23] to instrument CUDA kernels. NVBit enables inspection and modification of CUDA assembly code (SASS). We use NVBit's 'channel' (memory buffer) to communicate between the GPU and the CPU.

***Creating logical windows of memory instruction*:** First, ScopeAdvice instruments the kernel to divide the code into windows with fences as sentinels (Section V-B). A counter, called FenceId (initialized to 0), is incremented every time a fence is encountered in the code (static instruction). Each memory instruction (load, store, atomic) in the code is tagged with the *current* counter value (FenceId), creating logical windows of memory instructions. The counter value tagged to each memory instruction acts as its WindowId.

***Instrumenting for execution trace*:** ScopeAdvice instruments loads, stores, atomic RMWs, and fences in a kernel. Figure 7 depicts a unit of trace for memory instructions (load, stores, atomic RMWs). Besides the address and instruction type, ScopeAdvice captures the instruction's WindowId. Note that each static instruction may execute many times (*e.g.,* in a loop). Each such dynamic instance of an instruction will generate one unit of the trace. Recall that the trace should contain *only* global memory accesses. We use isspacep.global instruction to check if the address belongs to global memory. ScopeAdvice sets both the load and store bits in the trace for atomics and captures its scope. NVIDIA's compiler compiles loads and stores on 'volatile' data into distinguishable opcodes (*e.g.,* LDG.SYS), making them easily identifiable. The Scope field for loads and stores to 'volatile' data is set as 'system.'

The execution trace also tracks fences. However, tracking every dynamic instance of fence is unnecessary in practice. Memory instructions requiring synchronization and the corresponding fence often lie in the same code block (*i.e.,* inside the same conditional). Thus, when a thread executes a memory instruction, it also executes the fence. Thus, tracking which thread executed which fence is adequate. Whenever a fence is

executed, its static FenceId (assigned during instrumentation) and the thread identifier (ThreadId) that executed the fence are captured. ScopeAdvice keeps a two-dimensional bit vector indexed by FenceId and ThreadId. On encountering a fence during the kernel's execution, the corresponding bit in the bit vector is set (initially unset). Note that the dimensions of this bit vector are known before the kernel is launched. The number of unique FenceIDs is known during the instrumentation of the code, and the number of threads is a kernel launch parameter.

***Memory address metadata*:** Along with memory access trace and fences, we keep additional metadata for each unit of memory for ease of analysis. Figure 8 shows the layout of each unit of metadata for every 4Bytes of global memory. The Store bit indicates if the location was modified. The MultiBlock bit is set if multiple threadblocks access the location. BlockId keeps the threadblock identifier of the thread that last accessed the location. When a memory location is accessed for the first time, the corresponding Valid bit and BlockId field are set. The Store bit is set on *wr* or *atm*. On further accesses, if the BlockId of the accessing thread is different from the one in the metadata, the MultiBlock bit is set. Section VI-C details how the Count is used in an optimization.

***Managing memory for trace and metadata*:** ScopeAdvice ships trace units for memory accesses to the CPU as they are generated while the kernel executes. However, the trace units for fences and the memory metadata are shipped to the CPU after the kernel's execution ends. The size of the memory needed to hold the trace for fence and memory metadata is known before the kernel launch. As discussed earlier, the size of the two-dimensional bit vector of the fence trace is known apriori. The size of the memory metadata is related to the memory allocated for a kernel. CUDA programs typically pre-allocate memory (*e.g.,* using cudaMalloc) before the kernel launch. In contrast, the number of memory accesses is not known apriori. Thus, trace units are not allocated on the GPU but are shipped to the CPU for analysis as they are generated.

Reserving GPU memory for the fence trace and memory metadata would limit the memory available for the kernel. ScopeAdvice thus allocates memory using Unified Virtual Memory (UVM) [30], avoiding reservation. UVM driver automatically brings the portions of allocated data from the CPU onto the GPU memory as necessary upon GPU page faults.

B. Trace Aggregation and Metadata Analysis on the CPU

***Aggregating execution trace of memory instructions*:** As the instrumented kernel executes, memory access traces fill the buffer across the GPU and the CPU. ScopeAdvice employs CPU threads to move the buffer contents onto CPU memory. These threads also aggregate the trace units into a structure that maps an address accessed by threads to the operation (*rd*, *rdSkipL1*, *wr*, *wrV*, *atm*), scope (if applicable), WindowId, and the ThreadId to ease analysis. This continues till the kernel ends. The analysis of the trace starts after the kernel ends.

***Implementing getPrevSync and getNextSync*:** The two essential routines — getPrevSync (getNextSync) relate dynamic instances of memory instructions to the fences that determine the visibility of value consumed or produced by those instructions. The loads use getPrevSync, while stores use getNextSync. Figure 9 shows the pseudo-code of getPrevSync. It takes the threadId of the thread that executed the load and the windowId of that load. It typically returns the windowId argument decremented by 1. This value corresponds to the FenceId of the fence that affects the value read by that load.

```
1  def getPrevSync(threadId, windowId) -> fenceId:
2     fenceId = windowId - 1 # decrement for previous
3     while (fenceId > KERNEL_BEGINNING):
4        if (fenceTrace(threadId, fenceId)):
5           return fenceId
6        fenceId -= 1
7     return KERNEL_BEGINNING
```

Fig. 9: Simplified implementation of getPrevSync.

```
1  def detect(memTrace, memMetadata, fences) -> list[fenceId]:
2     osd = fences # All fences assumed over-synchronized
3     for (address : globalAddressesAllocated):
4        isSt = getBit(memMetadata[address], Store)
5        isMB = getBit(memMetadata[address], MultiBlock)
6        if (isSt and isMB):
7           for (I: memTrace[address]):
8              if (I.Load and not isVolatileLd(I)):
9                 # Correctly scoped, removing
10                osd.remove(getPrevSync(I.ThreadId, I.WindowId))
11             if (I.Store and not isDevAtomic(I)):
12                # Correctly scoped, removing
13                osd.remove(getNextSync(I.ThreadId, I.WindowId))
14    return osd  # over-synchronized fencesIds
```

Fig. 10: Detection logic that runs on the host.

While this is typically sufficient, the memory instruction and the identified fence may be present in different code blocks in some cases. For example, the memory instruction is present in the else block, and the identified fence is in the if block of a conditional. The thread would not have executed the fence as they are on different paths. In such cases, ScopeAdvice uses the fence execution trace to check if the thread executed the fence. Line 4 of Figure 9 checks if the given fenceId was executed by the thread (threadId). If not, it continues to decrement the counter until it finds a fence executed by the thread or the beginning of the kernel. In short, it identifies the latest fence executed by the GPU thread that determines the value the load observes. Similarly, getNextSync identifies the earliest fence after a store's execution, which guarantees the visibility of the value produced by that store.

***Analysis to detect over-synchronization*:** The analysis on the CPU starts after the kernel finishes execution, and all traces and metadata are available on the CPU. The detection logic starts by assuming *all* fences are over-synchronized. It then applies rules from Section V-C to establish the necessity of each fence or else declare the fence as over-synchronized.

Figure 10 shows the pseudo-code that identifies over-synchronized fences. It receives the memory instruction trace, the memory metadata, and the list of fence identifiers. It then iterates over all units of global memory addresses allocated by the kernel (globalAddressesAllocated). For every address, it consults the trace and the memory metadata. It checks if an address was written to and if it was accessed by multiple

threadblocks (line 6). If not, access to the given address does not necessitate a device-scoped fence (**Rule 2**). In line 7, it iterates over the aggregated execution trace units for all instructions that have accessed the given address. Lines 8 and 11 check for the first and second parts of **Rule 1**, respectively. If any of these conditions are satisfied, then the necessity of the corresponding *fence* is established, and the fence in question is not over-synchronized (lines 10 and 13). It uses the getPrevSync and getNextSync routines to find corresponding fences for a load and a store.

Finally, the tool is left with FenceIds of over-synchronized fences (if present). It then performs a source analysis to apply **Rule 3** to find redundant fences due to neighboring barriers. ScopeAdvice considers only the block-scoped fences or fences found over-synchronized in the previous step here.

***Actionable advice*:** ScopeAdvice reports over-synchronized and redundant fences with their source file, line number, and the reason (*i.e.,* **Variants 1, 2,** or **3**). The programmer can then improve the performance of their code by removing the reported over-synchronizations.

C. Optimizations to Reduce Overheads of ScopeAdvice

An obvious concern for any tool, even for a debugging tool like ScopeAdvice, is its runtime overheads. We apply *three* optimizations to limit its overheads.

***N-Buffering and Parallelism*:** ScopeAdvice employs multiple buffers and many analysis (CPU) threads to quickly empty the buffer between GPU and CPU onto the CPU's memory to avoid stalling the kernel. The detection routine is also multi-threaded. The for loop (line 3, Figure 10) iterates over all addresses accessed by the GPU threads.

***Execution sampling*:** While, in principle, the tool should track every load/store to global memory, it is often unnecessary in practice. An instruction in the code (static instance) may have many dynamic instances (*e.g.,* in a loop). It is natural that dynamic instances of a pair of static load/store issued by a pair of producer and consumer threads have the same traits. If one set of dynamic instances of a pair of static loads and stores are from threads within a threadblock, the other instances are likely to follow suit. Thus, ScopeAdvice always instruments the first dynamic instance of a given (static) memory instruction by each thread. After that, it randomly chooses an instance of the same instruction issued by the same thread with a probability $\frac{1}{Bound}$ (*Bound* is parameterized).

ScopeAdvice needs additional metadata for this. It maintains a 1Byte counter per thread, per static instruction, to track the dynamic instances of an instruction executed by a thread. Each time a thread executes an instruction, its counter is incremented. The dynamic instance is instrumented if the counter value equals a pre-assigned integer between 1 and Bound. Threads in each threadblock are assigned a random number between 1 and Bound beforehand.

***Filtering of memory access trace*:** Shipping trace units and analyzing them on the CPU adds large overheads. However, most of the trace units play little role in the analysis. For example, array elements are read once in the reduction kernel [16]. As the array is read-only, they cannot necessitate fences. However, this will not be known until the CPU analysis. Thus, the traces are wastefully shipped to the CPU for analysis.

ScopeAdvice addresses this problem by buffering a few units of traces on the GPU, anticipating that these traces will not be needed for the analysis. During the analysis on the CPU, when an address necessitates fences (*i.e.,* memory metadata points that an address is read and written to by multiple threadblocks), ScopeAdvice fetches trace units corresponding to that address for analysis. The trace units that are never fetched by the CPU are never analyzed by the CPU.

However, buffering the entire memory access trace on the GPU requires an unbounded amount of memory. ScopeAdvice bounds the number of trace units per address (4Bytes) that can reside on GPU. To keep a count, it uses the Count field (Figure 8) and maintains the trace units indexed by addresses. Count is incremented on access to an address. If Count is below a threshold (*inGPUTraces*), the generated trace unit is held on GPU memory. Otherwise, it is shipped to the CPU. During CPU analysis, ScopeAdvice fetches the buffered traces for an address only if necessary (line 6, Figure 10).

D. Use of Diverse Inputs and Fuzzer

As in any runtime tool, ScopeAdvice's advice is accurate for the given execution trace, *i.e.,* for an input and kernel's grid dimensions. While rare, a fence reported as over-synchronized for an execution trace may be correctly synchronized for a different execution trace. We recognize this as a *false positive*.

ScopeAdvice runs a kernel with multiple inputs to increase confidence in its advice, *i.e.,* eliminating false positives, a typical practice in software testing and dynamic analysis [31], [32]. It utilizes the test inputs shipped with the libraries and applications. We provide a wrapper script that runs ScopeAdvice against multiple inputs and reports an over-synchronization *only* if it manifests on executions with *every* input.

Further, ScopeAdvice can leverage automatic input generation techniques, such as evolutionary programming [33] and fuzzing [34]–[36]. We extended and integrated a fuzzer for GPU programs, CLFuzz [35], with ScopeAdvice to automatically generate inputs for CUDA kernels. CLFuzz is a coverage-guided fuzzer, *i.e.,* it generates inputs to exercise different paths through programs to eliminate false positives.

In Section VII, we show that the approach of using multiple inputs gives accurate advice, *i.e.,* the proposed optimization will not introduce bugs (*e.g.,* data races). We stress-tested ScopeAdvice against 25 kernels (*e.g.,* Rodinia [37], Polybench [38], and ScoR [16] benchmark suites) with and without over-synchronization and encountered *no* false positives – a testament to its robustness. Furthermore, a cautious programmer can run the optimized program containing ScopeAdvice's suggestions against race detectors too (*e.g.,* [15], [28], [39]).

E. Discussion: Why not static analysis?

It is natural to wonder if static analysis could detect over-synchronization to avoid the drawbacks of dynamic analysis,

TABLE I: Over-synchronization reported by ScopeAdvice. (V: Volatile, DA: Device-scoped atomic)

Application	Actual	Reported	Type
cuML (CU) [17]	3	3	**Variant** 1 (V)
String sort (ST) [19]	10	10	**Variant** 2
Compress (CP) [19]	2	2	**Variant** 2
Matrix multiplication (MM) [16]	3	3	**Variant** 1 (V), 3
Sgemm (GE) [41]	-	1	**Variant** 2
Unbalanced tree search (UT) [16]	2	2	**Variant** 1 (V)
Unbalanced tree search (UT-A) [16]	2	2	**Variant** 1 (DA)
Stencil (SC) [42]	2	2	**Variant** 3
Reduction (RD) [16]	0	0	-
Hash table (HS) [43]	0	0	-
Parallel linear recurrence (PL) [21]	0	0	-
Parallel merge (MG) [19]	0	0	-

i.e., the need for multiple inputs and runtime overheads. However, we found many reasons why static analysis is inadequate: ① index analysis techniques [40] for determining memory accesses fall short for kernels with complex access patterns (*e.g.,* input-dependent indexing), ② whether a load/store would access global memory is often known only during execution and ③ a fence may or may not execute based on the input, thus, known only at runtime. In these cases, static analysis becomes too conservative to yield helpful advice.

VII. Evaluation

We evaluate ScopeAdvice on an NVIDIA RTX 3090 GPU, with applications compiled using CUDA 11.2. The system has a 16-core CPU and 128GB of DRAM with NVIDIA driver v470. We use the buffer (channel) of 2MB between the GPU and the CPU, 768 (=N) buffers, 12 CPU threads, *Bound*=15 (sampling), and *inGPUTraces*=2 (trace filtering). These values are parameterized with defaults chosen empirically.

We answer the following questions: ① Can ScopeAdvice effectively identify over-synchronization? ② Do applications speed up by removing the identified over-synchronizations? ③ What are the runtime overheads of ScopeAdvice?

We use applications from popular libraries such as cuML [17], cudpp [19], cuBLAS [41], ScoR [16], KiloTM [43], and gpufilter [21]. Table I lists the different variants of over-synchronization in the applications we studied. We modify MM to include and evaluate **Variant 3** on an actual application. The unbalanced tree search (UT) application has **Variant 1** over-synchronization due to volatile variables. We then modified UT to evaluate **Variant 1** in the presence of device-scoped atomics (UT-A) since we did not find such cases of over-synchronization in the programs we studied. Except for UT-A, all applications having **Variant 1** are due to the presence of volatile variables. Note that the code for Stencil (SC) is not publicly available. We wrote the program using the pseudo-code available in the article (Figure 11 of [42]). To evaluate if ScopeAdvice reports false positives in the absence of over-synchronization, we evaluate hash table (HS) from KiloTM, reduction (RD) from ScoR, parallel linear recurrence (PL) from gpufilter, and parallel merge (MG) from cudpp. We also evaluated cuBLAS's sgemm (GE) to check if over-synchronization exists in hand-tuned, closed-source libraries.

TABLE II: Performance improvement by avoiding over-synchronization and reduction in stall cycles.

Application	Performance Improvements (%)	Stalls due to fences	
		Original	Modified
CU	37	24.8	14.5
ST	29	6.7	0
CP	0	0	0
MM	54	4.4	0.13
UT	17	2.3	0.8
UT-A	11	2.2	0.7
SC	50	2.8	0

We found that across all evaluated applications, at most two test inputs were enough to eliminate possible false positives.

A. Effectiveness of ScopeAdvice

Table I shows the number of cases of over-synchronization in each application (manually confirmed) and that reported by ScopeAdvice. It also shows the type of over-synchronization. We noticed that ScopeAdvice could find *all* the cases of over-synchronization without reporting any false positives.

We noticed that for ST, CP, SC, RD, HS, PL, and MG, reported cases of over-synchronization were the same across all tested inputs. However, we saw false positives for CU, MM, UT, and UT-A for *one* of their inputs, particularly the small input set. Smaller inputs require few threads for computation and exhibit little interaction amongst groups of threads (threadblock). Thus, fences were deemed over-synchronized. However, ScopeAdvice, with its integrated fuzzer, reports over-synchronization *only* when it appears for all inputs. Thus, it ultimately reported *no* false positives.

ScopeAdvice also reported 1 case of **Variant 2** in GE [41]. However, we could not validate it since the library is closed-source. Many such cases likely exist in closed-source libraries that could benefit from ScopeAdvice, too.

B. Application Performance Improvement

We measure speedups achieved by modifying the kernels as per the advice provided by ScopeAdvice. We execute two versions of each application: ① the original version, and ② the modified version without reported over-synchronization.

Table II reports the performance improvements, comparing the modified version with the original. We omit the applications that exhibit no over-synchronization. We report the mean performance improvement for each application. Following the tool's advice, applications sped up by 0-55%.

All applications except CP observe speedups. To better understand why different applications saw vastly different improvements, we use Nsight [44] profiling tool to analyze the GPU usage. We measure the average number of GPU stall cycles due to fences for each application. The stall cycles indicate the number of cycles for which the execution was stalled between issuing two consecutive warp (SIMD) instructions, on average. A higher number indicates a significant impact of fence instructions on an application's performance.

The last two sub-columns of Table II report the average stall cycles due to fences in the original application and the modified one, respectively. The stall cycles are significantly

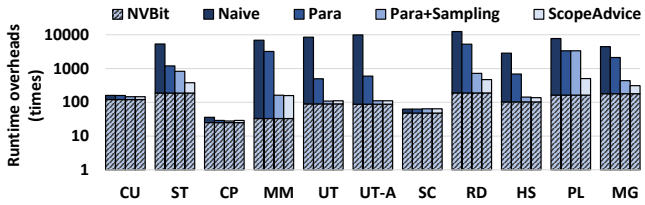

Fig. 11: Overheads of ScopeAdvice with each optimization compared to baseline (no instrumentation).

reduced in the modified versions for all applications except for CP. Since CP barely stalled due to fences to begin with, it is apparent why it did not witness any speedup.

C. Runtime Overheads of ScopeAdvice

ScopeAdvice is a (performance) debugging tool. Like any debugging tool, it adds runtime and memory overheads. The runtime overheads of ScopeAdvice stem from: ① instrumenting the kernel, ② executing the instrumented kernel on the GPU, ③ communicating the trace from the GPU to the CPU and ④ the time needed on the CPU to analyze the trace.

Figure 11 shows the tool's runtime overheads and the importance of the optimizations in reducing the overheads (lower is better). Each application has four bars, and each bar adds an optimization to the ones present in the bars to its left. We normalized the height of each bar to the runtime of the application executing without our tool. Notice the logarithmic y-axis. Each bar also has an NVBit component (bottom half). This component quantifies the contribution of the NVBit instrumentation library to the overheads. This component is the same across configurations for each application since the amount of instrumentation remains similar irrespective of optimizations. NVBit is the key contributor to the runtime overheads ($\approx$64% of end-to-end execution latency on average). This component is orthogonal to ScopeAdvice, and any future improvements in NVBit can reduce our tool's overheads.

The leftmost bar in each cluster, labeled **Naive**, represents the runtime without any optimization. It uses one buffer to communicate the traces from the GPU to the CPU and one CPU thread for analysis. Here, the CPU thread often fails to empty the buffer quickly enough, stalling the GPU kernel. Programs that generate many memory accesses and, thus, trace units in a short duration fill the buffer quickly, stalling the kernel often. The application RD suffered the most due to these stalls, witnessing an overhead of $12611\times$.

The bar labeled **Para** uses 12 analysis threads and 768 buffers for GPU-CPU communication. UT benefits the most from parallelism — its overheads dropped from $10349\times$ to $570\times$. However, CU is an exception here. In CU, only a leader thread among 256 threads of a threadblock writes the results of the threadblock's computation to the memory. Since the rate of accessing memory is relatively low, CU does not stall due to full buffers. It sees a slight increase in runtime due to the synchronization overheads among the CPU analysis threads.

TABLE III: ScopeAdvice's overheads (vs. application footprint) for metadata, fence, sampling and trace filter.

Applications	Metadata	Fence	Sampling	Trace filter
CU	1	15.9	144.06	2
ST	1	0.005	0.36	2
CP	1	0.0000002	0.00007	2
MM	1	0.002	0.06	2
UT	1	0.009	1.12	2
UT-A	1	0.009	1.12	2
SC	1	0.06	1.0002	2
RD	1	0.00001	0.006	2
HS	1	0.00007	0.002	2
PL	1	0.002	0.61	2
MG	1	0.000002	0.001	2

The bar labeled **Para+Sampling** represents the runtime with the optimizations from **Para** and **execution sampling**. Sampling reduces the amount of trace communicated to the CPU. Applications with high lock contention and memory operations in loops benefit the most from sampling. In particular, the number of times GPU-CPU communication occurred reduced by at least 90% for MM and UT compared to **Para**, lowering their overheads to $162\times$ and $113\times$, respectively.

The bar labeled **ScopeAdvice** represents the runtime with all the optimizations from Section VI-C. This further benefits from filtering memory access traces. The applications PL, ST, and RD benefited the most (improved by $6.8\times$, $2.2\times$, and $\times1.5$ over **Para+Sampling**). They had many sparsely accessed addresses that did not need fences. ScopeAdvice did not ship these trace units to CPU, reducing overheads by at least 35%. Finally, the tool's overheads stand at $146\times$ for CU, $431\times$ for ST, $29\times$ for CP, $158\times$ for MM, $116\times$ for UT, $111\times$ for UT-A, $65\times$ for SC, $499\times$ for RD, $137\times$ for HS, $522\times$ for PL, and $318\times$ for MG. The overheads include the time spent on the CPU to analyze the traces. The detection logic on the CPU accounts for up to 25% of the overheads (highest for ST).

While these overheads may look large, we put them in the context of other GPU dynamic analysis tools. A recent race detector, iGUARD [15], had overheads between $27\times$ (CP) to $649\times$ (PL). Another data race detector, BARRACUDA [28], reported runtime overheads of up to $\approx 3700\times$. Note that the goals of those tools are orthogonal to ScopeAdvice.

***Memory overheads*:** ScopeAdvice maintains 4Bytes of metadata for every 4Bytes of application memory on the GPU, adding a $1\times$ memory overhead. The memory required (in bits) to maintain the fence execution trace is the product of the number of GPU threads and the number of unique fences in the code. The memory overhead for the sampling optimization is the product of the number of GPU threads and the number of instrumented instructions. ScopeAdvice also maintains 2 (*#inGPUTraces*) * 4Bytes of memory for every 4Bytes of application memory on the GPU for the trace filtering optimization, adding a $2\times$ overhead.

Table III shows the memory overheads for each application for memory metadata, fence execution trace, execution sampling, and trace filtering. Overheads are calculated by normalizing the memory needed over the application's memory footprint. Typically, the trace filtering optimization is the

major contributor in most applications (2×). However, for CU, the fence execution trace and sampling metadata add large overheads. In CU, many threads perform loads and stores on a small memory region (0.49MB here). As many as 32678 threadblocks were launched with 256 threads each, with *nine* memory instructions and *eight* fence instructions. Though the absolute count of threads is large, all the threads cannot reside on the GPU simultaneously due to hardware constraints. The peak occupancy for CU is 492 threadblocks at any point in time. Therefore, the amount of memory that must be resident on the GPU for fence execution trace is 492 * 256 * 8 bits (≈0.12MB). Similarly, the amount required for sampling metadata is 492 * 256 * 9 Bytes (≈1.1MB). Thus, the memory required is relatively small (2.48× of the application footprint). Using UVM to allocate metadata ensures that *only* the required amount is present on the GPU.

VIII. Related Work

***Performance of GPU programs*:** Nsight [44] highlights program inefficiencies with profiling but provides minimal actionable advice to eliminate them. Recent studies overcome this limitation by developing tools that provide hints to improve program performance by eliminating value redundancies [45], [46], memory-use inefficiencies [47], GPU stall cycles [48]–[50], and redundant barriers [33]. These optimizations are complementary to ScopeAdvice. Orr *et al.* [51] proposed remote scope promotion to dynamically 'promote' scopes depending on the access-performing thread using modified hardware. ScopeAdvice is a software-only approach.

***Correctness of GPU programs*:** Race detection for GPU programs has been extensively studied in recent literature [15], [16], [28], [33], [39], [52]–[55]. These works aim to find bugs, which are cases of under-synchronization. ScopeAdvice performs an orthogonal task, *i.e.,* finding over-synchronization. Synccheck [56] aims to find incorrect use of barriers in CUDA programs, whereas ScopeAdvice aims to find inefficient choices of fence usage. Recent studies have aimed at finding memory-safety violations in GPU programs [57]–[59]. These bugs are orthogonal to the aim of ScopeAdvice.

IX. Conclusion

We show how CUDA programs do not fully harness scoped synchronization which leads to *over-synchronization*. We described *three* different variations of over-synchronization and measured their impact on performance. We built ScopeAdvice, a performance debugging tool that detects cases of over-synchronization in CUDA programs. Following the tool's advice, programs sped up by up to 55%.

Acknowledgment

We thank Ashish Panwar and Shweta Pandey for their feedback. We thank Kingshuk Majumder for discussions on compiler-based program analysis techniques. We gratefully acknowledge the Google Travel Award and IISc's Sarukkai Jagannathan Travel Award for enabling our participation in MICRO 2024. Ajay is supported by the Prime Minister's Fellowship Scheme for Doctoral Research, co-sponsored by the Confederation of Indian Industry, the Government of India, and Microsoft Research India. This work is partially supported by generous research grants from VMware Inc., AMD Inc., and Google faculty awards.

Appendix

A. Abstract

We provide the source code and setup of ScopeAdvice, our tool to identify over-synchronization. We include pre-compiled application binaries used in the evaluation. These binaries support NVIDIA Volta, Turing and Ampere-based GPUs. We include scripts to run and parse metrics used to generate the results in Section VII, *i.e.,* Tables I, II, III, and Figure 11.

B. Artifact check-list (meta-information)

- **Compilation:** CUDA 11.2, GCC 9.4.0, Ubuntu 20.04.
- **Binary:** Included for x86-64 host and NVIDIA Volta, Turing and Ampere GPUs.
- **Run-time environment:** Applications can be run on the bare machine. To run the applications, CUDA 11.2 is required on a Linux installation of Ubuntu 20.04.
- **Hardware:** An x86-64 machine with at least 16 cores (preferably AMD Ryzen 9 5950X) and 64GB of DDR4 DRAM, NVIDIA Ampere-based GPU (perferably RTX 3090), and PCIe 4.0 interconnect.
- **Execution:** The binaries are executed on a system with an attached NVIDIA GPU. Scripts are provided for the same.
- **Metrics:** Performance improvement, memory overhead, and runtime overhead.
- **Output:** We provide tabular data for every result in Section VII. Raw numbers for each experiment are present in the application folder corresponding to the experiment.
- **How much disk space required (approximately)?:** At least 2GB.
- **How much time is needed to complete experiments (approximately)?:** 14 hours.
- **Publicly available?:** Yes
- **Archived (provide DOI)?:** Yes, https://doi.org/10.5281/zenodo.13743937

C. Description

The artifact contains the source of ScopeAdvice along with the pre-compiled binaries of evaluated applications. It allows to reproduce the following results:

- Table I: Cases of over-synchronization reported by ScopeAdvice (Key result I).
- Table II: Performance improvement observed by eliminating over-synchronization (Key result II).
- Figure 11: Runtime overheads of using ScopeAdvice.
- Table III: Memory overheads of using ScopeAdvice.

1) How to access: The artifact is made available at https://github.com/csl-iisc/ScopeAdvice-MICRO24.git and https://doi.org/10.5281/zenodo.13743937.

2) Hardware dependencies: To evaluate ScopeAdvice, we recommend the following hardware configuration: ① NVIDIA Ampere-based GPU (preferably RTX 3090), ② x86-64 CPU with at least 16 cores (preferably AMD Ryzen 9 5950X), ③ PCIe 4.0×16 interconnect between the CPU and the GPU (for Unified Virtual Memory).

3) Software dependencies: The artifact needs CUDA-11.2 on a bare machine with Ubuntu 20.04. To install CUDA 11.2 on a bare machine, follow the instructions from NVIDIA. The artifact needs Docker installation to run our containerized setup. To install Docker on an Ubuntu machine, use:

```
$ sudo apt install docker.io
```

4) Data sets: Data generators are provided for few applications with large dataset requirement (*e.g.,* RD).

D. Installation

The artifact can be downloaded and accessed as:

```
$ git clone \
https://github.com/csl-iisc/ScopeAdvice-MICRO24.git
$ cd ScopeAdvice-MICRO24
```

E. Experiment workflow

The outermost directory of the artifact consists of three folders corresponding to the results in Section VII: table-1-and-3, table-2, and figure. table-1-and-3/ consists of all the application binaries and corresponding processing scripts to generate the results for Tables I and III. table-2/ consists of all the application binaries and corresponding processing scripts to generate the results for Table II. figure/ consists of all the application binaries and processing scripts to generate the results for Figure 11. Each folder has a script (`run.sh`) that generates the results for the corresponding experiment.

The experiments can be run within a docker container (advised) or a bare machine. Check README for further steps to setup docker environment for enabling GPU usage. To run experiments within the container, first build the container in the outermost directory as follows:

```
$ docker build . -t sa:v1
```

Run the docker container in interactive mode using the following command. This command opens a bash shell in the docker image.

```
$ docker container run -it \
--runtime=nvidia --gpus all sa:v1 bash
```

To run all the experiments we provide a single script (`run.sh`) in the outermost directory of the artifact. This script executes the run scripts in the folders mentioned above. This script must be run as follows:

```
$ ./run.sh
```

Individual experiments can be run from the corresponding experiments folder as follows:

```
$ cd table-1-and-3;./run.sh;cd ..
$ cd figure;./run.sh;cd ..
$ cd table-2;./run.sh;cd ..
```

F. Evaluation and expected results

Observations for each experiment in Section VII is printed on the terminal and a comma separated file is generated in the corresponding experiments folder. The generated results can be compared against tables and figures reported in the paper. The generated results can be viewed in the following paths:

```
$ vi table-1-and-3/table-1-result.csv
$ vi table-1-and-3/table-3-result.csv
$ vi figure/result.csv
$ vi table-2/result.csv
```

The results of runtime overheads (Figure 11) can vary across runs (due to a comparatively small baseline). The results will be within error margin (check `get-error.py`). The trends of different optimization levels will remain the same ir-respective of this variance.

G. Notes

If running on a non-docker setup, ensure that fetching GPU performance counters is enabled for all the users before running the scripts used to reproduce results from Table II. NVIDIA Nsight requires sudo privilege to fetch GPU performance counters. This can be disabled with modifications to linux configuration files [60].

The pre-compiled binaries support NVIDIA Volta, Turing and Ampere-based GPUs. On all the GPUs, observations in Table I remain the same. However, other results (namely Table II) will change based on the GPU.

REFERENCES

[1] Google, "Cloud gpus," 2019. [Online]. Available: https://cloud.google.com/gpu/

[2] Amazon, "P3 instances with v100," 2020. [Online]. Available: https://aws.amazon.com/ec2/instance-types/p3/

[3] NVIDIA, "Gpus everywhere," 2019. [Online]. Available: https://blogs.nvidia.com/blog/2017/05/08/microsoft-azure-gpu-instances/

[4] SHI, "Nvidia a100 80gb pcie gpu," 2021. [Online]. Available: https://www.shi.com/product/41094090/NVIDIA-Tesla-A100-GPU-computing-processor

[5] SHI, "Nvidia dgx station a100," 2021. [Online]. Available: https://www.shi.com/product/41820041/NVIDIA-DGX-Station-A100

[6] J. Dean, "Google ai chief jeff dean interview: Machine learning trends in 2020," 2019. [Online]. Available: https://venturebeat.com/2019/12/13/google-ai-chief-jeff-dean-interview-machine-learning-trends-in-2020//

[7] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, "AntMan: Dynamic scaling on GPU clusters for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.

[8] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019.

[9] M. Zhao, N. Agarwal, A. Basant, B. Gedik, S. Pan, M. Ozdal, R. Komuravelli, J. Pan, T. Bao, H. Lu, S. Narayanan, J. Langman, K. Wilfong, H. Rastogi, C.-J. Wu, C. Kozyrakis, and P. Pol, "Understanding data storage and ingestion for large-scale deep recommendation model training: Industrial product," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022.

[10] Z. Cai, Q. Zhou, X. Yan, D. Zheng, X. Song, C. Zheng, J. Cheng, and G. Karypis, "Dsp: Efficient gnn training with multiple gpus," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2023.

[11] Y. Gao, Y. He, X. Li, B. Zhao, H. Lin, Y. Liang, J. Zhong, H. Zhang, J. Wang, Y. Zeng, K. Gui, J. Tong, and M. Yang, "An empirical study on low gpu utilization of deep learning jobs," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, apr 2024, pp. 880–880. [Online]. Available: https://doi.ieeecomputersociety.org/

[12] NVIDIA, "Parallel thread execution isa," 2021. [Online]. Available: https://docs.nvidia.com/cuda/parallel-thread-execution/

[13] J. Sanders and E. Kandrot, "Cuda by example: Errata page," 2021. [Online]. Available: https://developer.nvidia.com/cuda-example-errata-page

[14] C. Flanagan and S. N. Freund, "Fasttrack: Efficient and precise dynamic race detection," in *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2009.

[15] A. K. Kamath and A. Basu, "Iguard: In-gpu advanced race detection," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021.

[16] A. K. Kamath, A. A. George, and A. Basu, "Scord: A scoped race detector for gpus," in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, 2020.

[17] S. Raschka, J. Patterson, and C. Nolet, "Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence," *arXiv preprint arXiv:2002.04803*, 2020.

[18] NVIDIA, "Cuda c++ programming guide," 2022. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[19] cudpp, "Cuda data parallel primitives library," 2021. [Online]. Available: https://github.com/cudpp/cudpp

[20] NVIDIA, "Basic linear algebra on nvidia gpus," 2022. [Online]. Available: https://developer.nvidia.com/cublas

[21] D. Nehab, A. Maximo, R. S. Lima, and H. Hoppe, "gpufilter - gpu recursive filtering," 2016. [Online]. Available: https://github.com/andmax/gpufilter

[22] D. Lustig, S. Sahasrabuddhe, and O. Giroux, "A formal analysis of the nvidia ptx memory consistency model," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019.

[23] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, "Nvbit: A dynamic binary instrumentation framework for nvidia gpus," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019.

[24] M. D. Sinclair, J. Alsop, and S. V. Adve, "Efficient gpu synchronization without scopes: Saying no to complex consistency models," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015.

[25] J. Alsop, M. S. Orr, B. M. Beckmann, and D. A. Wood, "Lazy release consistency for gpus," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.

[26] J. Alglave, M. Batty, A. F. Donaldson, G. Gopalakrishnan, J. Ketema, D. Poetzl, T. Sorensen, and J. Wickerson, "Gpu concurrency: Weak behaviours and programming assumptions," in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2015.

[27] J. Sanders and E. Kandrot, "Cuda by example: An introduction to general-purpose gpu programming," 2021. [Online]. Available: https://developer.nvidia.com/cuda-example

[28] A. Eizenberg, Y. Peng, T. Pigli, W. Mansky, and J. Devietti, "Barracuda: Binary-level analysis of runtime races in cuda programs," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017.

[29] S. Biswas, M. Zhang, M. D. Bond, and B. Lucia, "Valor: Efficient, software-only region conflict exceptions," in *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 2015.

[30] NVIDIA, "Unified memory for cuda beginners," 2017. [Online]. Available: https://devblogs.nvidia.com/unified-memory-cuda-beginners/

[31] M. Zalewski, "American fuzzy lop - whitepaper," 2016. [Online]. Available: https://lcamtuf.coredump.cx/afl/technical_details.txt

[32] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.

[33] M. Wu, Y. Ouyang, H. Zhou, L. Zhang, C. Liu, and Y. Zhang, "Simulee: Detecting cuda synchronization bugs via memory-access modeling," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020.

[34] H. Chen, S. Guo, Y. Xue, Y. Sui, C. Zhang, Y. Li, H. Wang, and Y. Liu, "MUZZ: Thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020.

[35] C. Peng and A. Rajan, "Automated test generation for opencl kernels using fuzzing and constraint solving," in *Proceedings of the 13th Annual Workshop on General Purpose Processing Using Graphics Processing Unit*, 2020.

[36] W. Li, Z. Chen, X. He, G. Duan, J. Sun, and H. Chen, "Cvfuzz: Detecting complexity vulnerabilities in opencl kernels via automated pathological input generation," *Future Generation Computer Systems*, 2022.

[37] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009.

[38] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to gpu codes," in *2012 Innovative Parallel Computing (InPar)*, 2012.

[39] Y. Peng, V. Grover, and J. Devietti, "Curd: A dynamic cuda race detector," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2018.

[40] R. Alur, J. Devietti, O. S. Navarro Leija, and N. Singhania, "Gpudrano: Detecting uncoalesced accesses in gpu programs," in *Computer Aided Verification*, 2017.

[41] NVIDIA, "Cuda samples," 2021. [Online]. Available: https://github.com/NVIDIA/cuda-samples

[42] S. Dey, M. Mukul, P. Sharma, and S. Biswas, "Predictive data race detection for gpus," *CoRR*, 2021. [Online]. Available: https://arxiv.org/abs/2111.12478

[43] W. W. L. Fung, I. Singh, A. Brownsword, and T. M. Aamodt, "Hardware transactional memory for gpu architectures," in *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2011.

[44] NVIDIA, "Nsight compute - an interactive kernel profiler for cuda applications," 2021. [Online]. Available: https://developer.nvidia.com/nsight-compute

[45] K. Zhou, Y. Hao, J. Mellor-Crummey, X. Meng, and X. Liu, "Valueexpert: Exploring value patterns in gpu-accelerated applications," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022.

[46] K. Zhou, Y. Hao, J. Mellor-Crummey, X. Meng, and X. Liu, "Gvprof: A value profiler for gpu-based clusters," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[47] M. Lin, K. Zhou, and P. Su, "Drgpum: Guiding memory optimization for gpu-accelerated applications," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023.

[48] K. Zhou, X. Meng, R. Sai, and J. Mellor-Crummey, "Gpa: A gpu performance advisor based on instruction sampling," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[49] D. Shen, S. L. Song, A. Li, and X. Liu, "Cudaadvisor: Llvm-based runtime profiling for modern gpus," in *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, 2018.

[50] L. Braun and H. Fröning, "Cuda flux: A lightweight instruction profiler for cuda applications," in *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2019.

[51] M. S. Orr, S. Che, A. Yilmazer, B. M. Beckmann, M. D. Hill, and D. A. Wood, "Synchronization using remote-scope promotion," in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2015.

[52] NVIDIA, "Racecheck tool," 2021. [Online]. Available: https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html#racecheck-tool

[53] M. Zheng, V. T. Ravi, F. Qin, and G. Agrawal, "Grace: A low-overhead mechanism for detecting data races in gpu programs," in *Proceedings of the 16th ACM Symposium on Principles and Practice of Parallel Programming*, 2011.

[54] A. Betts, N. Chong, A. Donaldson, S. Qadeer, and P. Thomson, "Gpuverify: A verifier for gpu kernels," in *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, 2012.

[55] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan, "Gklee: Concolic verification and test generation for gpus," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2012.

[56] NVIDIA, "Synccheck tool," 2021. [Online]. Available: https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html#synccheck-tool

[57] M. Tarek Ibn Ziad, S. Damani, A. Jaleel, S. W. Keckler, and M. Stephenson, "Cucatch: A debugging tool for efficiently catching memory safety violations in cuda applications," *Proc. ACM Program. Lang.*, 2023.

[58] C. Erb, M. Collins, and J. L. Greathouse, "Dynamic buffer overflow detection for gpgpus," in *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2017.

[59] B. Di, J. Sun, D. Li, H. Chen, and Z. Quan, "Gmod: A dynamic gpu memory overflow detector," in *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, 2018.

[60] NVIDIA, "Nvidia permission issue with performance counters," 2018. [Online]. Available: https://developer.nvidia.com/ERR_NVGPUCTRPERM